

Arduino Cookbook : De la Théorie à la Pratique

Chapitre 5 : Entrées Numériques et Analogiques Simples

Présenté par : Dr B. DIOP

29 mars 2026

Référence : Michael Margolis, *Arduino Cookbook*, 2nd Edition, O'Reilly

5.0 Introduction : Écouter le monde physique

De l'action à la perception

Jusqu'à présent, nous avons surtout ordonné à l'Arduino de faire des choses (allumer une LED). Il est temps de lui donner des "sens" pour qu'il puisse réagir à son environnement.

Dans ce chapitre, nous allons apprendre à :

- Lire des états "Tout ou Rien" (Boutons, interrupteurs).
- Gérer le bruit électrique (Broches flottantes) et le rebond mécanique.
- Lire des valeurs continues (Potentiomètres, capteurs de lumière).
- Convertir une échelle de valeurs (map).
- Lisser les données pour éliminer l'instabilité des capteurs.

5.1 Les Entrées Numériques (Digital Input)

Une entrée numérique ne comprend que deux états logiques :

- **HIGH (Haut)** : Tension proche de 5V (représente souvent 1, Vrai ou ON).
- **LOW (Bas)** : Tension proche de 0V (représente souvent 0, Faux ou OFF).

La fonction clé : `digitalRead(broche)` Cette fonction interroge le microcontrôleur et renvoie instantanément HIGH ou LOW selon la tension présente sur la broche.

5.2 Code : Lire un bouton-poussoir simple

Voici le code minimal pour lire l'état de la broche 2 et l'afficher sur l'ordinateur.

```
const int boutonPin = 2; // Broche ou est connecte le bouton
int etatBouton = 0;      // Variable pour stocker le resultat

void setup() {
  Serial.begin(9600);    // Initialise la communication avec le PC

  // Configure la broche 2 comme une ENTREE
  pinMode(boutonPin, INPUT);
}

void loop() {
  etatBouton = digitalRead(boutonPin); // Lecture de la broche

  if (etatBouton == HIGH) {
    Serial.println("Le bouton est PRESSE !");
  } else {
    Serial.println("Le bouton est RELACHE.");
  }
  delay(100); // Petit delai pour ne pas saturer l'ecran
}
```

5.3 Le problème du "Flottement" (Floating Pin)

Le piège du bouton-poussoir

Si vous connectez un bouton entre le 5V et la broche 2, que se passe-t-il quand le bouton n'est PAS pressé ? La broche 2 n'est connectée à rien !

Une broche d'Arduino "en l'air" (non connectée) agit comme une antenne. Elle capte les ondes électromagnétiques ambiantes (Wi-Fi, secteur 220V). La fonction `digitalRead()` renverra des séries de HIGH et de LOW totalement aléatoires.

La solution : Il faut "forcer" un état par défaut quand le bouton est ouvert. On utilise pour cela une **Résistance de Tirage (Pull-up ou Pull-down)**.

5.4 Résistances : Pull-Down vs Pull-Up

1. La résistance Pull-Down (Tirage vers le bas) :

- Une résistance (ex : 10 k Ω) relie la broche au GND (0V).
- Bouton relâché = 0V (LOW).
- Bouton pressé (connecté au 5V) = Le courant passe, la broche lit 5V (HIGH).

2. La résistance Pull-Up (Tirage vers le haut) :

- Une résistance relie la broche au 5V.
- Bouton relâché = 5V (HIGH).
- Bouton pressé (connecté au GND) = Le courant fuit vers la masse, la broche lit 0V (LOW).

La logique est inversée !

5.5 L'astuce Arduino : INPUT_PULLUP

L'ATmega328P possède des résistances Pull-Up internes (environ 20 k Ω) déjà intégrées au silicium ! Vous n'avez pas besoin d'ajouter de résistance matérielle sur votre circuit.

```
const int boutonPin = 2;

void setup() {
  Serial.begin(9600);
  // Active la resistance interne vers le 5V
  pinMode(boutonPin, INPUT_PULLUP);
}

void loop() {
  int etat = digitalRead(boutonPin);

  // ATTENTION a la logique inversee !
  if (etat == LOW) {
    Serial.println("Bouton PRESSE (relie a la masse)");
  } else {
    Serial.println("Bouton RELACHE (tire vers le 5V)");
  }
}
```

5.6 Le phénomène de Rebond (Bouncing)

Les contacts métalliques à l'intérieur d'un interrupteur ne se ferment pas instantanément. Sous l'effet du ressort, les lamelles rebondissent l'une contre l'autre pendant quelques millisecondes.

Le résultat vu par l'Arduino : Si l'Arduino est assez rapide, pour une seule pression humaine, il lira : LOW – HIGH – LOW – HIGH – LOW avant de se stabiliser.

Cela peut déclencher votre code 5 fois au lieu d'une ! Il faut filtrer ce bruit : c'est le **Debouncing** (Anti-rebond).

5.7 Code : L'Anti-Rebond Logiciel (Debounce)

L'idée est de ne valider le changement d'état que s'il reste stable pendant un certain temps (ex : 50 ms).

```
const int boutonPin = 2;
int etatPrecedent = HIGH; // Rappel : on utilise INPUT_PULLUP
unsigned long tempsDernierRebond = 0;
unsigned long delaiRebond = 50; // 50 millisecondes

void setup() { pinMode(boutonPin, INPUT_PULLUP); Serial.begin(9600); }

void loop() {
    int lecture = digitalRead(boutonPin);

    // Si l'état change (même à cause d'un rebond), on réinitialise le chrono
    if (lecture != etatPrecedent) { tempsDernierRebond = millis(); }

    // Si l'état est stable depuis plus de 50ms, c'est une vraie pression
    if ((millis() - tempsDernierRebond) > delaiRebond) {
        if (lecture == LOW) {
            Serial.println("Vraie pression validée !");
        }
    }
    etatPrecedent = lecture;
}
```

5.8 Les Entrées Analogiques (Analog Input)

Le monde n'est pas binaire. La température, la lumière ou la position d'un joystick varient de manière continue.

Le Convertisseur Analogique/Numérique (ADC) : L'Arduino Uno possède 6 broches analogiques (A0 à A5). Elles intègrent un ADC (Analog to Digital Converter) de **10 bits**.

- **10 bits** signifie qu'il peut découper une tension en $2^{10} = 1024$ **paliers**.
- 0V renverra la valeur 0.
- 5V renverra la valeur 1023.
- 2.5V renverra la valeur 512.

5.9 Code : Lire un Potentiomètre

Un potentiomètre (ou diviseur de tension) permet de faire varier la tension entre 0V et 5V envoyée sur la broche A0.

Note : Les broches analogiques sont des entrées par défaut, pas besoin de `pinMode()`.

```
const int potPin = A0; // Broche connectee au curseur central du potentiometre

void setup() {
  Serial.begin(9600);
}

void loop() {
  // Lit la tension et la convertit en un nombre entre 0 et 1023
  int valeurBrute = analogRead(potPin);

  // Optionnel : Reconvertir en Volts reels pour l'affichage (Regle de 3)
  float voltage = valeurBrute * (5.0 / 1023.0);

  Serial.print("Valeur ADC : ");
  Serial.print(valeurBrute);
  Serial.print(" | Tension : ");
  Serial.println(voltage);
}
```

5.10 La Fonction `map()` : Changer d'échelle

Vous lisez un potentiomètre (**0 à 1023**), mais vous voulez utiliser cette valeur pour régler la luminosité d'une LED via PWM, qui n'accepte que des valeurs de **0 à 255**.

La solution mathématique : `map()` Cette fonction convertit proportionnellement une valeur d'une plage à une autre.

```
valeurConvertie = map(valeurInitiale, minIn, maxIn, minOut, maxOut);
```

5.11 Code : Utiliser map() pour la luminosité

```
const int potPin = A0;    // Entree analogique
const int ledPin = 9;     // Sortie PWM

void setup() {} // Rien a initialiser pour A0, pinMode automatique

void loop() {
  int lecturePot = analogRead(potPin); // Donne un nombre de 0 a 1023

  // Transforme l'echelle : 1023 devient 255, 512 devient 127...
  int luminosite = map(lecturePot, 0, 1023, 0, 255);

  // Applique la nouvelle valeur au materiel
  analogWrite(ledPin, luminosite);

  delay(10);
}
```

Note pour cet exemple précis : Diviser simplement lecturePot / 4 fonctionnerait aussi, mais map() est plus flexible (ex : inverser les valeurs avec map(val, 0, 1023, 255, 0)).

5.12 Le Bruit Analogique

Lorsque vous lisez un capteur sensible (comme une photorésistance ou un capteur de température LM35), vous constaterez que la valeur fluctue constamment : 512, 514, 511, 513, 515...

Causes du bruit :

- Interférences électromagnétiques ambiantes.
- Qualité de l'alimentation 5V (qui sert de référence à l'ADC).
- Limites de précision de l'ADC interne.

Solution : Ne jamais utiliser la valeur brute directement, mais réaliser une **moyenne glissante (Smoothing)**.

5.13 Code : Lissage Analogique (Moyenne)

L'algorithme consiste à accumuler plusieurs lectures successives et à les diviser par le nombre d'échantillons pour obtenir une valeur stable.

```
const int capteurPin = A0;
const int nbEchantillons = 10; // Plus ce nombre est grand, plus c'est stable

void setup() { Serial.begin(9600); }

void loop() {
    long somme = 0; // 'long' evite le debordement si la somme depasse 32767

    // Prend 10 mesures tres rapides
    for (int i = 0; i < nbEchantillons; i++) {
        somme += analogRead(capteurPin);
        delay(2); // Petit delai pour laisser l'ADC respirer
    }

    int valeurLisee = somme / nbEchantillons; // Calcule la moyenne

    Serial.println(valeurLisee); // La valeur affichee sera tres stable !
    delay(100);
}
```

Résumé du Chapitre 5

- **Broches flottantes** : Ne laissez jamais une broche d'entrée sans référence de tension. Utilisez systématiquement `pinMode(pin, INPUT_PULLUP)`.
- **Anti-Rebond** : Le matériel n'est pas parfait. Prévoyez toujours une logique temporelle (`millis()`) pour ignorer les contacts parasites des interrupteurs.
- **Analogique** : L'ADC convertit de 0 à 5V en un entier de 0 à 1023. La broche A0 n'a pas besoin d'être déclarée dans le `setup()`.
- **Fiabilité des données** : Les capteurs du monde réel sont bruyants. L'utilisation de `map()` pour recadrer et des moyennes (Boucle `for`) pour lisser sont des étapes obligatoires en robotique.

Fin du Chapitre 5. Vos étudiants peuvent désormais créer des interfaces réactives et interpréter les données analogiques de leur environnement !